Pontificia Universidad Católica de Chile
Instituto de Ingeniería Matemática y Computacional 2026-2

IMT2220

Cálculo para Ciencia de Datos

Tarea Computacional

Profesor
Eduardo Cerpa
eduardo.cerpa@uc.cl

Ayudante
Kevin Cardemil
kevin.cardemil@uc.cl

TAREA COMPUTACIONAL

Introducción

Esta tarea consta de dos problemas en los que se aplican las herramientas de optimización vistas en el curso (puntos críticos, matriz Hessiana y descenso del gradiente) a dos situaciones propias de la Ciencia de Datos:

- **Problema 1: Reconstrucción de una imagen.** A partir de una fotografía desenfocada y con ruido se busca recuperar la imagen original, planteando la reconstrucción como un problema de minimización. Se calculará la solución directa y se estudiará por qué es inestable, para luego compararla con la reconstrucción obtenida mediante la regularización de Tikhonov.
- **Problema 2: Red neuronal con dos capas ocultas.** Se entrenará, mediante descenso del gradiente, una red neuronal pequeña que aproxime una función de una variable a partir de datos. Para ello se calculará el gradiente de la función de pérdida con la regla de la cadena, se estudiará la sensibilidad del entrenamiento y se analizarán los límites de lo que esta red puede representar.

La tarea debe resolverse en los grupos asignados. Para cada problema se les entregarán archivos con datos.

Indicaciones para el código

- Todo el código debe desarrollarse en Python, en Jupyter notebooks (`.ipynb`).
- Deben programar ustedes mismos todas las funciones involucradas (funciones de costo, gradientes, métodos de descenso del gradiente, la red neuronal, etc.); no pueden usar comandos ya creados que resuelvan lo pedido.
- **Está permitido** usar operaciones básicas de `numpy` (crear arreglos, sumas, productos `@`, `reshape`, `np.sqrt`, `np.exp`, `np.log`, `np.linalg.norm`, generación de números aleatorios), funciones para leer los datos (`np.load`, `np.loadtxt` o `pandas.read_csv`) y `matplotlib` para graficar.
- **No está permitido** usar funciones que resuelvan sistemas lineales, inviertan matrices, calculen valores propios o resuelvan problemas de mínimos cuadrados u optimización (por ejemplo `np.linalg.solve`, `np.linalg.inv`, `np.linalg.pinv`, `np.linalg.lstsq`, `np.linalg.eig`, `scipy.optimize` o `sklearn`), ni librerías de redes neuronales o de diferenciación automática (por ejemplo `torch`, `tensorflow`, `keras` o `jax`). Las únicas excepciones son las que se indican explícitamente en el enunciado.

Instrucciones de entrega

Cada grupo debe entregar el **JUEVES 3 de NOVIEMBRE vía CANVAS** los siguientes 5 archivos:

- Un PDF con el desarrollo y las respuestas de ambos problemas.
- Dos Jupyter notebooks (`.ipynb`), uno para cada problema, con el código utilizado, ejecutados de principio a fin.
- Dos PDF, uno por cada Jupyter notebook renderizado, en el que se vean el código y todas sus salidas (resultados y gráficos).

> **Cómo obtener el PDF de un Jupyter notebook.** En VS Code, ejecuten todas las celdas del notebook y expórtenlo como `.html` (menú `···` de la barra superior del notebook → *Exportar* → *HTML*). Luego abran el archivo `.html` en su navegador e imprímanlo como PDF (*Imprimir* → *Guardar como PDF*).

PROBLEMA 1

Reconstrucción de una imagen

Una imagen en escala de grises de $N \times N$ píxeles es una matriz $X \in M_{N\times N}(\mathbb{R})$, donde cada coeficiente de X pertenece al intervalo $[0, 1]$ siendo 0 el negro y 1 el blanco. Si la recorremos por filas obtenemos un vector

$$x = (X_{11}, X_{12}, \ldots, X_{1N}, X_{21}, \ldots, X_{2N}, \ldots, X_{N1}, \ldots, X_{NN}) \in \mathbb{R}^n, \quad n = N^2.$$

Una cámara desenfocada no registra x, sino

$$b = Ax + \eta,$$

donde $A \in M_{n\times n}(\mathbb{R})$ modela el desenfoque y $\eta \in \mathbb{R}^n$ es ruido del sensor. La matriz A es elegida para que cada píxel registrado b_p sea un promedio ponderado de los píxeles de la imagen x en una ventana de 9×9 centrada en p (los píxeles que caen fuera de la imagen cuentan como negros), más el ruido aportado por η. Además, cada entrada de A es no negativa, es decir,

$$a_{pq} \geq 0 \quad \text{para cada fila } p \text{ y columna } q \text{ de } A,$$

y cada fila y cada columna de A suma a lo más 1. Debido a lo anterior, el desenfoque nunca aumenta la norma:

$$\|Ax\|_2 \leq \|x\|_2 \quad \text{para todo } x \in \mathbb{R}^n.$$

En el archivo `datos_P1.npz` encontrarán la matriz A y el vector b (que pueden cargar con `datos = np.load("datos_P1.npz")` y obtener como `datos["A"]` y `datos["b"]`) correspondientes a una imagen de 64×64 píxeles ($n = 4096$), pero no la imagen x. El objetivo de este problema es reconstruirla. Además:

- las entradas de η son independientes, con media 0 y desviación estándar $\sigma = 0.01$, por lo que $\|\eta\|_2 \approx \delta := \sigma\sqrt{n} = 0.64$;
- A^TA es simétrica. También, denotamos por $\mu_1, \ldots, \mu_n$ sus valores propios y por $v_1, \ldots, v_n$ sus vectores propios asociados, ortonormales. El menor de ellos es $\mu_{\min} \approx 2 \cdot 10^{-16}$, y 1858 de los 4096 son menores que 10^{-6}.

Para ver un vector $v \in \mathbb{R}^n$ como imagen pueden usar `plt.imshow(v.reshape(64, 64), cmap="gray", vmin=0, vmax=1)`. Excepcionalmente, en este problema pueden usar `np.linalg.solve` para resolver los sistemas lineales que aparecen en (b), (c) y (d).



Figura 1: Imagen observada b (el vector b reordenado como matriz de 64×64).

(a) **Mínimos cuadrados.** Considere $J_0(x) = \|Ax - b\|_2^2$.

i. Calcule $\nabla J_0(x)$ y la matriz Hessiana H_{J_0}. Deduzca que los puntos críticos de J_0 son las soluciones de las *ecuaciones normales* $A^T Ax = A^T b$.

ii. Muestre que H_{J_0} es semidefinida positiva y que todo punto crítico de J_0 es un mínimo global (como J_0 es un polinomio de grado 2, su aproximación de Taylor de segundo orden es exacta).

iii. Finalmente, muestre que, si todos los valores propios de $A^T A$ son positivos, entonces J_0 tiene un único minimizador, y que, si $A^T A$ tiene un valor propio nulo, entonces J_0 tiene infinitos minimizadores.

(b) **Reconstrucción.** Como todos los μ_i son positivos, por (a) J_0 tiene un único minimizador x_0, que satisface las ecuaciones normales y que en este caso coincide con la solución de $Ax = b$.

i. **Implementación.** Calcule x_0 resolviendo $Ax = b$ y muestre lado a lado las imágenes b y x_0. Calcule $\|x_0\|_2$, $\max_i |x_{0,i}|$ y el residuo $\|Ax_0 - b\|_2$. ¿x_0 se parece a b? ¿Se parece a una imagen? (Recuerde que los píxeles de la imagen verdadera están en $[0, 1]$, por lo que su norma es a lo más $\sqrt{n} = 64$).

ii. **Explicación.** Sabemos que $b = Ax^* + \eta$, donde x^* es la imagen verdadera. Muestre que

$$\langle x_0 - x^*, v_i \rangle = \frac{\langle Av_i, \eta \rangle}{\mu_i} \quad \text{y que} \quad |\langle x_0 - x^*, v_i \rangle| \leq \frac{\|\eta\|}{\sqrt{\mu_i}}.$$

En este contexto, el término $\langle Av_i, \eta \rangle / \mu_i$ corresponde al error que el ruido produce en la dirección v_i. Explique por qué. Luego, usando que $\mu_{\min} \approx 2 \cdot 10^{-16}$ y que muchos μ_i son menores que 10^{-6}, explique por qué un ruido pequeño ($\|\eta\|_2 \approx 0.64$) puede producir un error tan grande en x_0 como el observado en i.

(c) **Regularización de Tikhonov (método *ridge*).** Para $\lambda > 0$ considere

$$J_\lambda(x) = \|Ax - b\|_2^2 + \lambda \|x\|_2^2.$$

i. Calcule ∇J_λ y H_{J_λ}. Muestre que todos los valores propios de H_{J_λ} son mayores o iguales que 2λ, y concluya que J_λ tiene un único minimizador *global*, dado por

$$x_\lambda = (A^T A + \lambda I)^{-1} A^T b.$$

ii. Sabemos que $b = Ax^* + \eta$, donde x^* es la imagen verdadera. Muestre que

$$\langle x_\lambda, v_i \rangle = \frac{\mu_i}{\mu_i + \lambda} \langle x^*, v_i \rangle + \frac{\langle Av_i, \eta \rangle}{\mu_i + \lambda} \quad \text{y que} \quad \left| \frac{\langle Av_i, \eta \rangle}{\mu_i + \lambda} \right| \leq \frac{\|\eta\|}{2\sqrt{\lambda}},$$

sin importar cuán pequeño sea μ_i. En este contexto, el primer término de la igualdad corresponde a la parte de la imagen verdadera x^* que se recupera, y el segundo, al aporte del ruido. Explique por qué. *Indicación:* Puede utilizar que $\mu + \lambda \geq 2\sqrt{\mu\lambda}$ para cualquier $\mu \geq 0$.

iii. A partir de lo anterior, interprete: ¿qué ocurre con las componentes de x^* con $\mu_i \gg \lambda$ y con aquellas con $\mu_i \ll \lambda$? ¿Qué espera obtener si λ es muy pequeño? ¿Y si es muy grande?

iv. **Implementación.** Calcule x_λ resolviendo $(A^T A + \lambda I)x = A^T b$ para $\lambda \in \{10^{-3.5}, 10^{-2.5}, 1\}$. Muestre en una misma figura las imágenes b, x_0 y las tres reconstrucciones x_λ, y reporte para cada una de ellas su norma y su residuo $\|Ax - b\|_2$. Compare las reconstrucciones con x_0 y explique lo que observa usando ii. y iii. ¿Cuál de los tres valores de λ entrega la mejor reconstrucción?
Indicación: no tiene sentido exigir que Ax se ajuste a b mejor que el tamaño del ruido, por lo que conviene comparar cada residuo con $\delta = 0.64$.

(d) **Píxeles dañados.** Supongamos ahora que cerca del 10 % de los píxeles del sensor están dañados: registran 0 o 1 sin importar la imagen. En `datos_P1.npz` encontrarán una segunda observación $\tilde{b}$ (`datos["b_def"]`) tomada con ese sensor. Reconstruya la imagen a partir de $\tilde{b}$ con la regularización de Tikhonov y $\lambda = 10^{-2.5}$, como en (c) iv. Muestre lado a lado $\tilde{b}$ y la reconstrucción. Para entender por qué fracasa:

i. Si escribimos $\tilde{b} = Ax^* + \tilde{\eta}$, el término $\tilde{\eta}$ corresponde al ruido de esta observación. Muestre que $\tilde{\eta} = \eta + (\tilde{b} - b)$ e interprete cada uno de sus dos sumandos.

ii. Calcule $\|\tilde{b} - b\|_2$ y compare $\|\tilde{\eta}\|_2$ con δ.

iii. Usando (c) ii. con $\tilde{\eta}$ en lugar de η, explique por qué la reconstrucción fracasa. ¿Bastaría con aumentar λ? Relacione su respuesta con lo observado en (c) iv.

> **(BONUS) Mínimos cuadrados ponderados.** Si se sabe qué píxeles están dañados, se pueden ignorar esas lecturas. Sea $w \in \{0,1\}^n$ con $w_p = 0$ si el píxel p está dañado y $w_p = 1$ si no. En vez de J_λ se minimiza
>
> $$J_W(x) = \sum_{p=1}^{n} w_p \big((Ax)_p - \tilde{b}_p\big)^2 + \lambda\|x\|_2^2 = \big(Ax - \tilde{b}\big)^T W \big(Ax - \tilde{b}\big) + \lambda\|x\|_2^2, \quad W = \operatorname{diag}(w).$$
>
> Con este cambio la reconstrucción queda prácticamente igual a la obtenida sin píxeles dañados, porque cada píxel de la imagen influye en hasta 81 lecturas del sensor y perder un 10 % de ellas deja información de sobra. Implemente esta modificación del algoritmo.

PROBLEMA 2

Red neuronal con dos capas ocultas

Este problema se trata de entrenar una red neuronal que pueda aproximar una función de una variable $f\colon [-1,1] \to \mathbb{R}$. En el archivo que se les hará llegar encontrarán una lista de datos de entrenamiento de la forma $(x_i, y_i) \in \mathbb{R}^2$ con $i = 1, 2, \ldots, N$, en donde $x_i \in [-1,1]$ es la entrada e y_i es el valor observado de $f(x_i)$, con un pequeño error de medición.

Usaremos la estructura de la Figura 2, que tiene **dos capas ocultas con una sola neurona cada una**: la entrada x pasa por una primera neurona, cuya salida h_1 es la entrada de una segunda neurona, cuya salida h_2 determina la predicción $\hat{y}$ de la red. Denotando por σ una función de activación tendremos

$$h_1 = \sigma(a_1 x + b_1), \qquad h_2 = \sigma(a_2 h_1 + b_2), \qquad \hat{y} = c + v h_2,$$

en donde los seis parámetros de ajuste de la Red Neuronal son los pesos a_1, a_2, v y los sesgos b_1, b_2, c. La neurona de salida no aplica la función de activación. Agruparemos los parámetros en el vector

$$\theta = (a_1, b_1, a_2, b_2, v, c) \in \mathbb{R}^6$$

y escribiremos $\hat{y} = \hat{y}(x;\theta)$ para enfatizar que la salida de la red depende tanto de la entrada como de los parámetros.

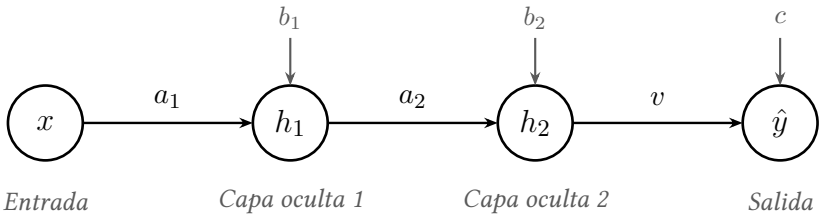


Figura 2: Estructura de la Red Neuronal. Las etiquetas sobre las flechas negras indican los pesos y las flechas grises, los sesgos de cada neurona. Las neuronas ocultas aplican σ; la de salida es lineal.

(a) Considere la función de activación sigmoide

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

Muestre que su derivada satisface $\sigma'(t) = \sigma(t)(1 - \sigma(t))$ y deduzca que $0 < \sigma'(t) \leq \frac{1}{4}$ para todo $t \in \mathbb{R}$. ¿En qué punto σ' alcanza su máximo?

(b) Considerando los datos de entrenamiento $(x_i, y_i) \in \mathbb{R}^2$, construya la función de pérdida

$$J(\theta) = \frac{1}{2N} \sum_{i=1}^{N} \big(\hat{y}(x_i;\theta) - y_i\big)^2$$

y calcule explícitamente su gradiente $\nabla_\theta J(\theta) \in \mathbb{R}^6$ en un punto cualquiera $\theta \in \mathbb{R}^6$, es decir, sus seis derivadas parciales.

Sugerencia: Para cada dato (x_i, y_i) considere las cantidades intermedias

$$z_{1,i} = a_1 x_i + b_1, \quad h_{1,i} = \sigma(z_{1,i}), \quad z_{2,i} = a_2 h_{1,i} + b_2,$$
$$h_{2,i} = \sigma(z_{2,i}), \quad \hat{y}_i = c + v h_{2,i}, \quad e_i = \hat{y}_i - y_i,$$

y aplique la regla de la cadena siguiendo el camino por el cual cada parámetro afecta a la pérdida $L_i = \frac{1}{2} e_i^2$ del dato i. Por ejemplo, el peso a_1 afecta a L_i a través de

$$a_1 \longrightarrow z_{1,i} \longrightarrow h_{1,i} \longrightarrow z_{2,i} \longrightarrow h_{2,i} \longrightarrow \hat{y}_i \longrightarrow L_i.$$

Note que hay factores repitidos en varias derivadas y úselos para escribir el gradiente de forma compacta.

(c) Programe funciones que, dado θ, calculen $J(\theta)$ y $\nabla_\theta J(\theta)$. Inicialice los pesos y sesgos de esta Red Neuronal asignándole a cada componente de un $\theta = \theta_0$ un valor en el intervalo $[-1, 1]$ y calcule $J(\theta_0)$. Luego, verifique su cálculo del gradiente comparando cada derivada parcial con la aproximación por diferencias centradas

$$\frac{\partial J}{\partial \theta^{(j)}}(\theta_0) \approx \frac{J(\theta_0 + h\mathbf{e}_j) - J(\theta_0 - h\mathbf{e}_j)}{2h} =: D_j, \quad j = 1, \ldots, 6,$$

con $h = 10^{-5}$, donde $\theta^{(j)}$ es la j-ésima componente de θ y $\mathbf{e}_j$ es el j-ésimo vector canónico de $\mathbb{R}^6$. Reporte el máximo error absoluto obtenido,

$$E = \max_{j=1,\ldots,6} \left| \frac{\partial J}{\partial \theta^{(j)}}(\theta_0) - D_j \right|.$$

(d) Para la búsqueda de un mínimo programe un algoritmo de descenso del gradiente

$$\theta_{k+1} = \theta_k - \alpha \nabla J(\theta_k)$$

con un paso fijo $\alpha = 1$ y genere 20.000 iteraciones. Muestre con un gráfico la evolución del error $J(\theta_k)$ en función de k y comente su forma.

(e) Entregue los valores encontrados para $\theta_{20.000} \in \mathbb{R}^6$ y el valor $J(\theta_{20.000})$. Grafique en conjunto los datos de entrenamiento y la curva $x \mapsto \hat{y}(x; \theta_{20.000})$ en el intervalo $[-1, 1]$.

(f) Estudie la sensibilidad del entrenamiento:

i. Partiendo del mismo θ_0, repita (d) con pasos $\alpha \in \{0{,}01; 0{,}1; 1; 2{,}5\}$ y grafique las cuatro curvas de error en un mismo gráfico. Comente qué ocurre cuando el paso es muy pequeño o muy grande.

ii. Repita (d) con $\alpha = 1$ para 10 inicializaciones distintas con componentes en $[-1, 1]$ y otras 10 con componentes en $[-5, 5]$. ¿Se alcanza siempre el mismo valor final de J? Explique lo observado usando el comportamiento de $\sigma'(t)$ cuando $|t|$ es grande, y comente si el descenso del gradiente garantiza encontrar el mínimo global de J.

(g) En un segundo archivo se les entregan datos $(x_i, \tilde{y}_i) \in \mathbb{R}^2$, con $i = 1, 2, \ldots, N$, generados del mismo modo que los anteriores, pero a partir de otra función $\tilde{f}\colon [-1, 1] \to \mathbb{R}$.

i. Entrene la misma red con estos datos, partiendo del mismo θ_0 de (c), con $\alpha = 1$ y 20.000 iteraciones. Grafique los datos junto a la curva obtenida y compárela con la de (e). ¿Logra la red ajustar estos datos?

ii. Muestre que, para cualquier $\theta \in \mathbb{R}^6$, la función $x \mapsto \hat{y}(x; \theta)$ es monótona en $\mathbb{R}$ y que sus valores están siempre entre c y $c + v$. A partir de esto y de lo que observa en los datos, explique por qué esta red no puede aproximar bien a $\tilde{f}$ en el intervalo $[-1, 1]$, sin importar cuántas iteraciones de entrenamiento se realicen ni qué paso o inicialización se use.